

Index B+Trees

Index B+Tree

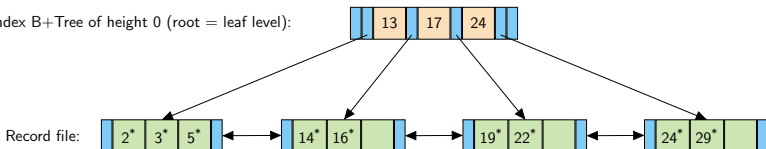
There are two sub-variants of the Index B+ depending on the sort order (if any) of the data records in the record file:

- **Secondary Index B+Tree:** Here the records are **NOT** necessarily sorted in the data file of blocks. Thus each entry in the leaf nodes of the tree contains a pair consisting of a key value and a disk address which identifies the disk block where the record with that key value can be found. Leaf nodes also have forward and reverse pointers.
- **Primary Index B+Tree:** Here the records are kept sorted by the key value used in the B+Tree in the data file containing the blocks of records. Thus the leaf nodes of the B+tree only need to store discriminator values to separate the data file blocks and look much like internal B+Tree nodes except that they also have forward and reverse pointers.

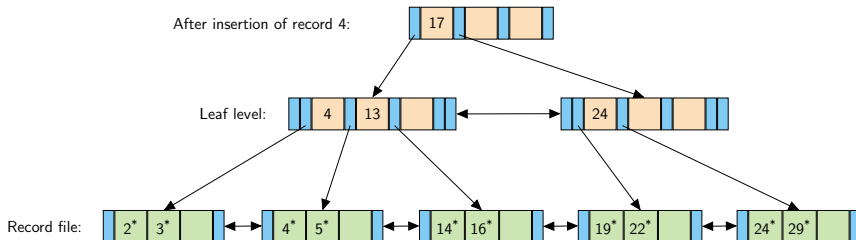
There can be only one primary key index on a file of records as the records can be in only one order, but there can be multiple secondary indexes on the same file.

Primary Index B+Tree

A Primary Index B+Tree of height 0 (root = leaf level):



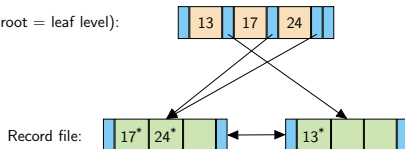
After insertion of record 4:



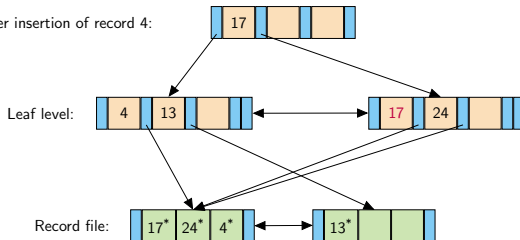
Note that the leaf level of the Primary B+Tree contains discriminators to separate blocks of records in the record file

Secondary Index B+Tree

A Secondary Index B+Tree of height 0 (root = leaf level):



After insertion of record 4:



Note that every value in the record file has an individual entry in the leaf level of the secondary B+Tree, hence we see key values in the leaf level that may occur higher in the tree, e.g. 17 in the above case.